VERILOG · HOMEWORK · HW6

Button-controlled 10000 Counter

100 MHz vs 1 MHz Debounce Sampling · 3 Input Scenarios

button_debouncer / FSM_RUN_STOP / counter_datapath_v2 / FND scan



정안성 · 2026.10.13

Contents

01 **Design** p.3 – 8

Goal · Board I/O · Block Diagram · FSM · Debouncer · Sim 설정

02 **Scenario 1 · 정상 입력** p.9 – 16

RUN · STOP · MODE(DOWN) · CLEAR 파형

03 **Scenario 2 · 입력 노이즈** p.17 – 21

Chatter · 100 MHz vs 1 MHz sampling 비교

04 **Frequency Domain** p.22 – 27

Impulse train · Burst spectrum · |H(f)| · Aliasing

05 **Scenario 3 · 버튼 우선순위** p.28 – 35

동시 입력 a ~ f · Skew · 전체 run 비교

06 **Conclusion** p.36 – 37

결론 · 고찰 (TB feedback 포함)

Three Goals: Full Design, Sampling Comparison, Scenario Analysis

01

01

전체 설계

- 버튼 3개 입력 10000진 카운터
- button_debouncer → FSM_RUN_STOP
- → counter_datapath_v2 → FND scan
- RUN · STOP · CLEAR · MODE (UP / DOWN)

02

Sampling 비교

- 버튼 입력 100 MHz sampling : window 80 ns
- 버튼 입력 1 MHz sampling : window 8 µs
- 동일 입력 → 반응 시간 · 노이즈 내성 비교
- 차이를 주파수 도메인으로 해석

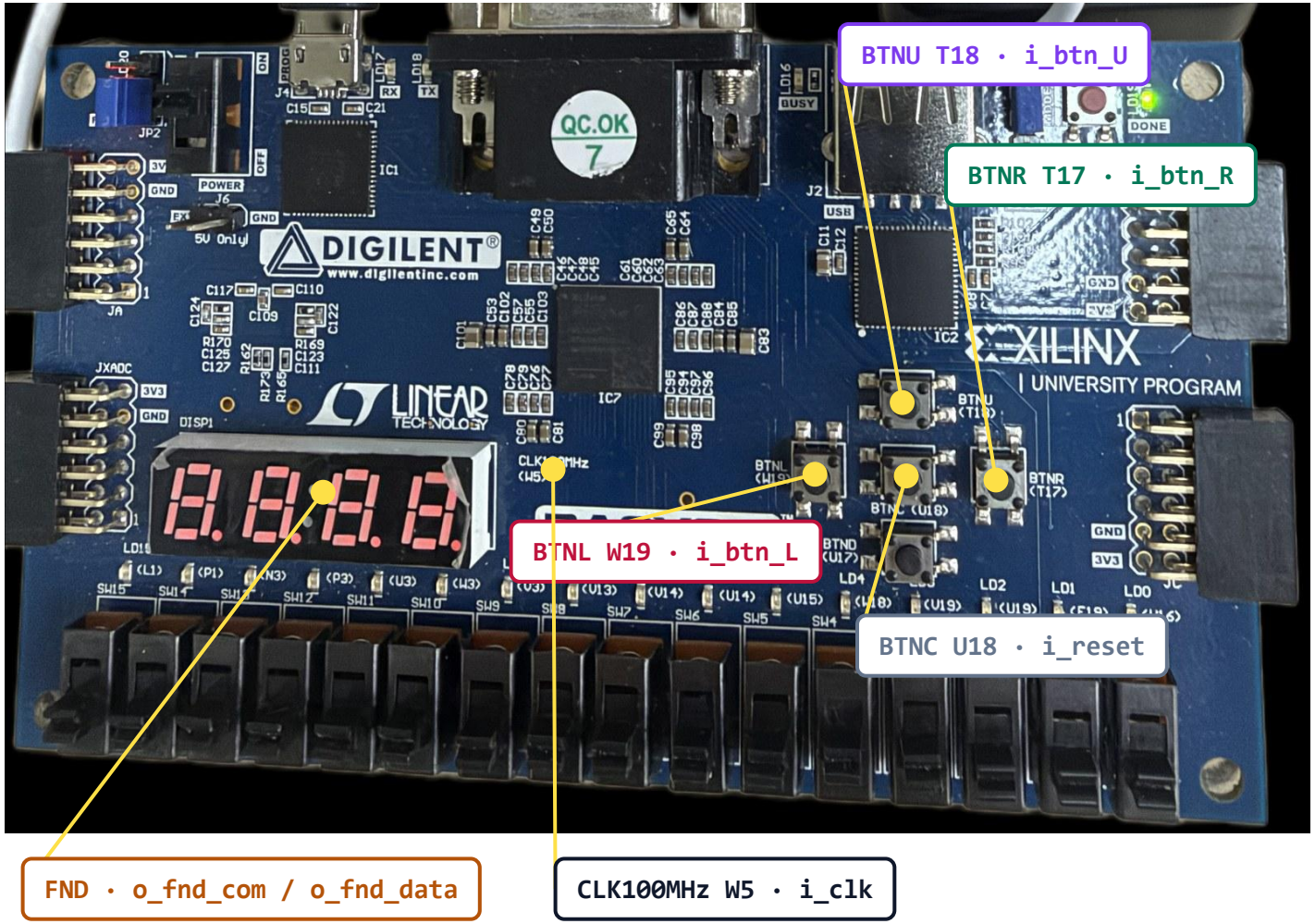
03

시나리오 분석

- 1 정상 입력 : state 전환 · UP / DOWN · CLEAR
- 2 입력 노이즈 : 80 ns ~ 8 µs burst
- 3 동시 입력 : 버튼 우선순위 R > L > U
- 시나리오별 Vivado 파형 + 동작 근거

Basys 3: Three Buttons In, Four-digit FND Out

01

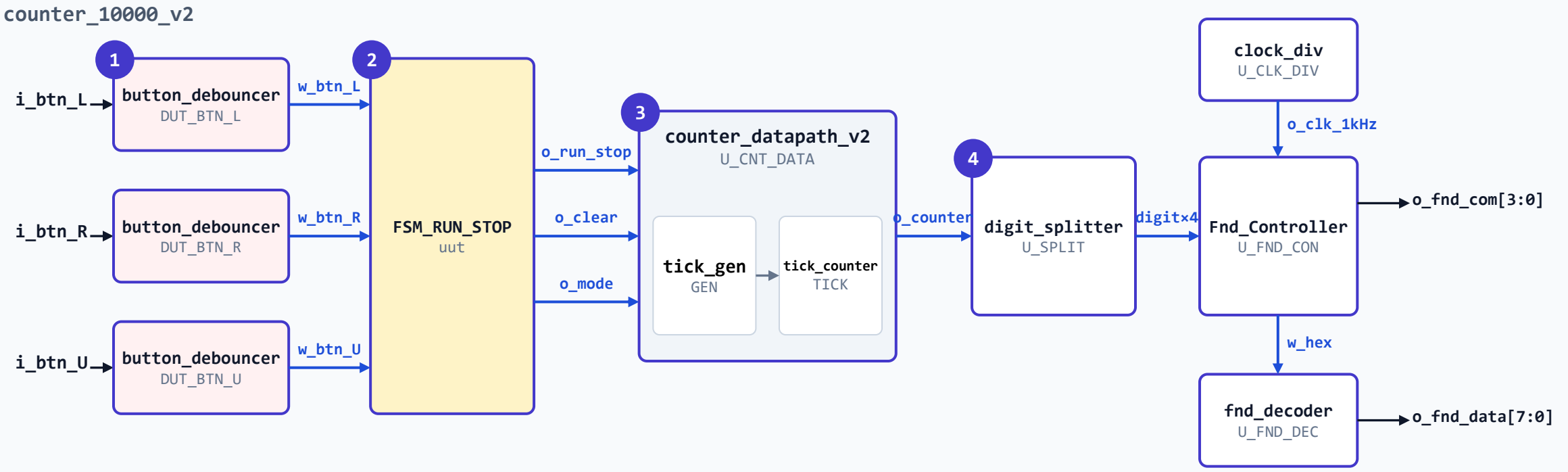


FND_BTN.xdc

Port	Pin	역할
i_clk	W5	100 MHz system clock
i_reset	U18	BTNC · 전체 reset
i_btn_R	T17	BTNR · RUN / STOP
i_btn_L	W19	BTNL · CLEAR
i_btn_U	T18	BTNU · MODE (UP / DOWN)
o_fnd_com[3:0]	U2 U4 V4 W4	AN0 ~ AN3 · digit 선택
o_fnd_data[7:0]	W7 W6 U8 V8 U5 V5 U7 V7	CA ~ CG, DP · segment

counter_10000_v2: Debounce, Decide, Count, Display

01



i_clk 100 MHz · i_reset → 전 module 공통 (clock_div · Fnd_Controller = inverter 경유 active-low reset)

1 **button_debouncer ×3** 8 sample 연속 1 → 1 clock tick

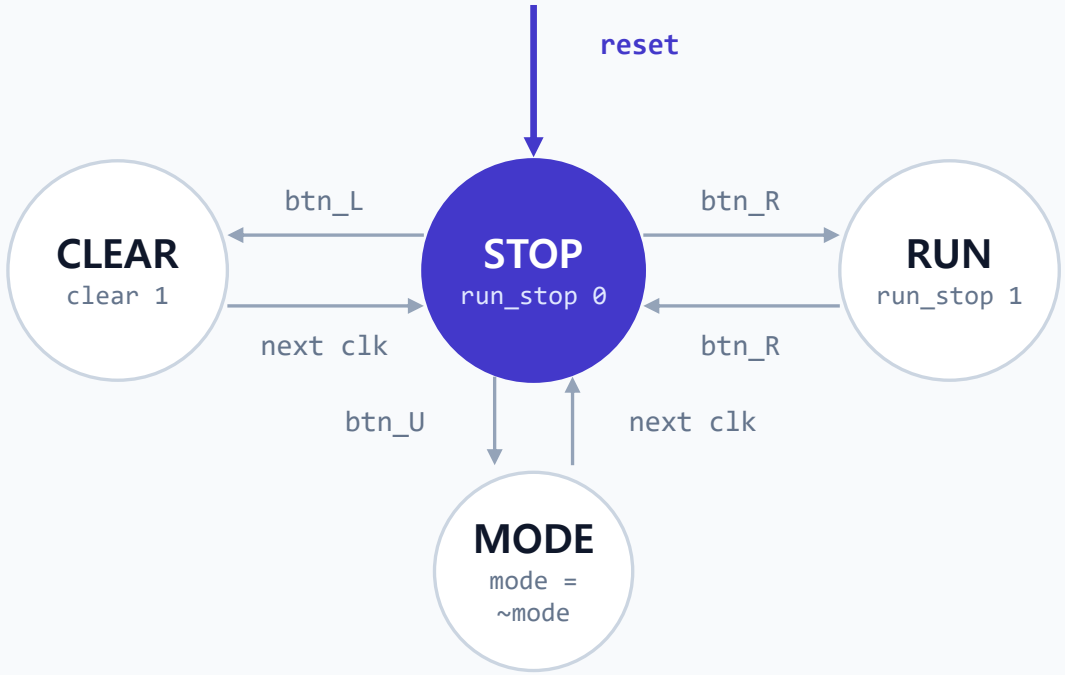
2 **FSM_RUN_STOP** tick → STOP · RUN · CLEAR · MODE

3 **counter_datapath_v2** tick_gen 주기 tick → 0 ~ 9999 UP / DOWN

4 **FND path** digit 분리 → scan → segment decode

FSM_RUN_STOP: STOP Is the Hub, Priority R > L > U

01



Moore 출력 (state만으로 결정) · enum: CLEAR 0 · STOP 1 · RUN 2 · MODE 3

State	출력	다음 State
STOP	run_stop 0 · clear 0	btn_R → RUN btn_L → CLEAR btn_U → MODE
RUN	run_stop 1	btn_R → STOP btn_L · btn_U 무시
CLEAR	clear 1 (1 clock)	다음 clock → STOP
MODE	mode_next = ~mode_reg	다음 clock → STOP (mode_reg 갱신)

1. **STOP 우선순위** if btn_R → else if btn_L → else if btn_U
2. **reset** state = STOP, mode_reg = 0 (UP)
3. **mode_reg** register → MODE 다음 clock edge에서 반전

1 MHz Debouncer Samples Every 100 Clocks, Window Grows to 8 µs

01

button_debouncer · 100 MHz sampling



button_debouncer_1MHz · 1 MHz sampling



항목	button_debouncer (100 MHz)	button_debouncer_1MHz	근거
Sample 주기	10 ns	1 µs (100 clock)	q_reg shift clock
Window (8 sample)	80 ns	8 µs	8 연속 1 조건
인정 경계 (sim sweep)	70 ns 이하 거부 · 80 ns 이상 인정	7.0 µs 이하 거부 · 8.0 µs 이상 인정	사이 구간 = 입력 위상 의존
입력 → state (sim 실측)	85 ns	8,005 ns	Scenario 1 BTN_R

Tick and FND Scan Scaled 1/100 Together, Debouncer Kept Real

01

항목	실제 HW	Simulation
i_clk	100 MHz	100 MHz
tick (tick_gen FCOUNT)	10,000,000 clk = 100 ms	100,000 clk = 1 ms
FND scan (clock_div)	1 kHz · 1 ms / digit	100 kHz · 10 µs / digit
tick : scan 1회전 (4 digit)	100 ms : 4 ms = 25 : 1	1 ms : 40 µs = 25 : 1
button_debouncer	80 ns / 8 µs window	동일 (축소 없음)

1. **축소 이유** tick 100 ms 그대로일 경우 10 tick = 1 s = 1억 clock
2. **같은 비율** tick · scan 동시 1/100 → scan 25회전 / count, HW 동일
3. **Debouncer** 입력 sampling 현상 관찰 대상 → 실제 속도 유지

tb_counter_scaled

driver(i)
1 : 정상 입력 · 2 : 고정 noise table · 3 : 동시 입력

↓

counter_10000_v2_scaled
`define DEB_1MHZ → button_debouncer_1MHz
없을 경우 → button_debouncer (100 MHz)

↓

$display
cnt_R · cnt_L · cnt_U (tick 수) · state · o_counter

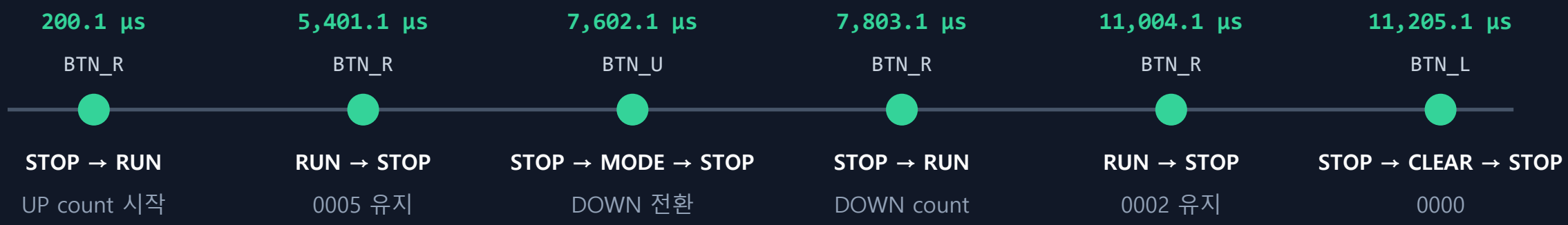
press() 폭 HOLD_NS = 1 µs × DEB_SCALE (100 MHz : 1, 1 MHz : 100)

02

Scenario 1 · 정상 입력

Scenario 1: Clean Presses Visit Every State

driver(1) · 100 MHz debouncer · press 폭 1 µs

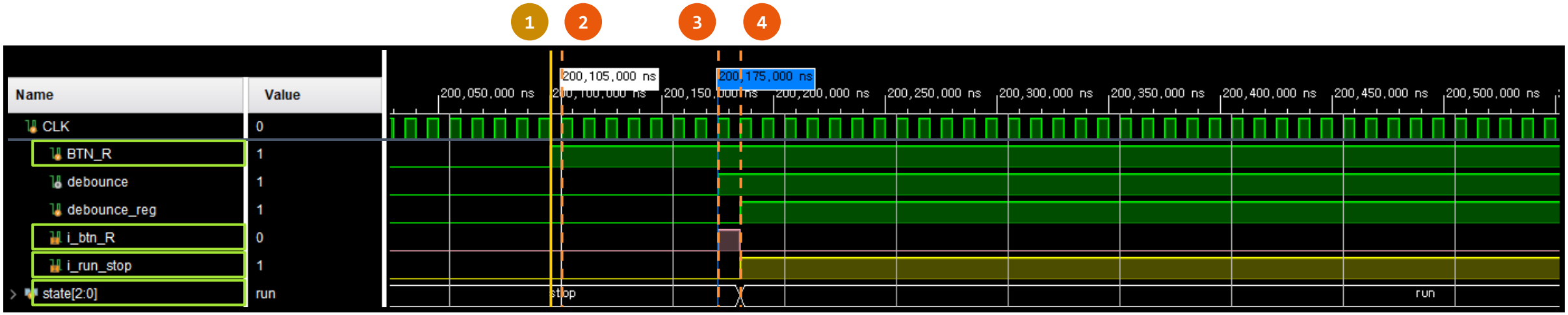


o_counter : 0000 → 0005 (UP 5 tick) → 0002 (DOWN 3 tick) → 0000 (CLEAR)

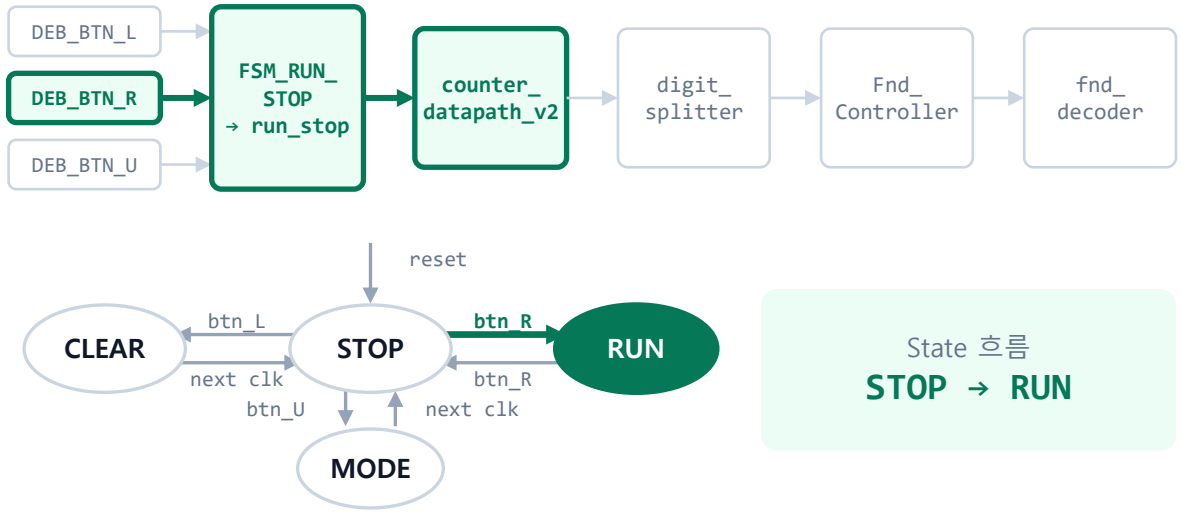
100 MHz debouncer

02

BTN_R Moves STOP to RUN 80 ns After the First Sample



1. **200,100 ns** BTN_R 입력 (posedge 사이)
2. **200,105 ns** 첫 sample → q_reg[7] = 1
3. **200,175 ns** 8번째 sample → debounce · i_btn_R tick
4. **200,185 ns** STOP → RUN, i_run_stop = 1

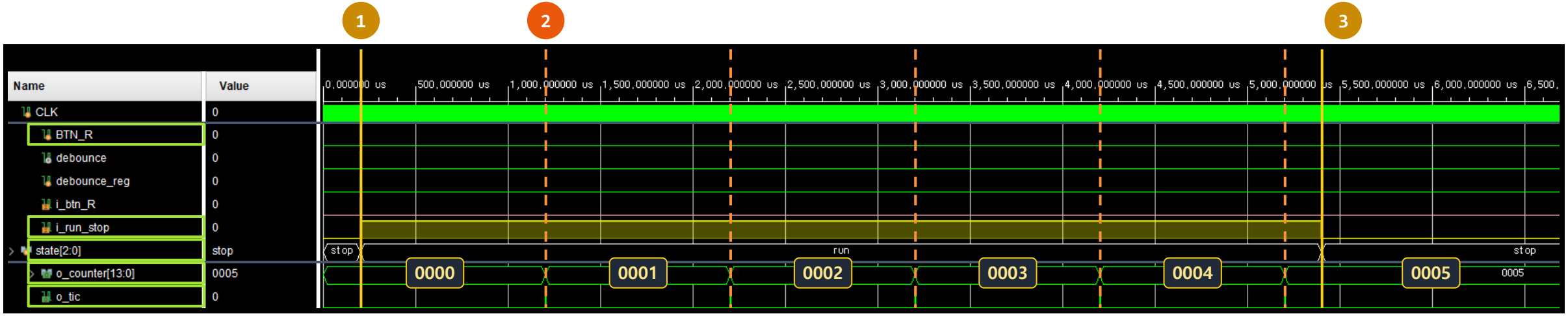


State 흐름
STOP → RUN

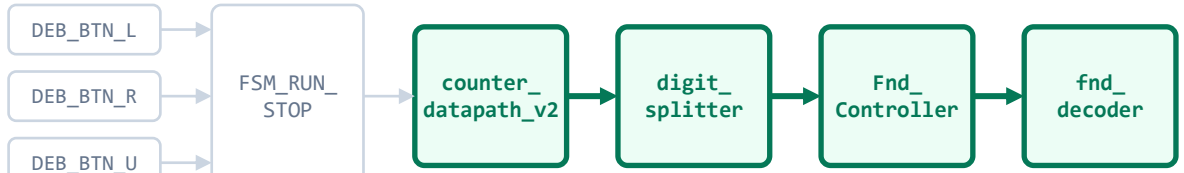
100 MHz debouncer

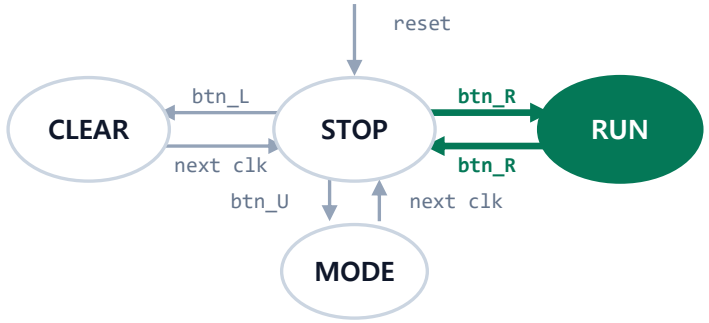
02

RUN Adds One Count per 1 ms Tick up to 0005



1. **200 µs** BTN_R → RUN
2. **1.2 ~ 5.2 ms** 1 ms마다 o_tic → o_counter +1 (UP)
3. **5,401 µs** 두 번째 BTN_R → STOP, 0005 유지
4. **tick 1 ms** FCOUNT 100,000 clock (HW 100 ms의 1/100)



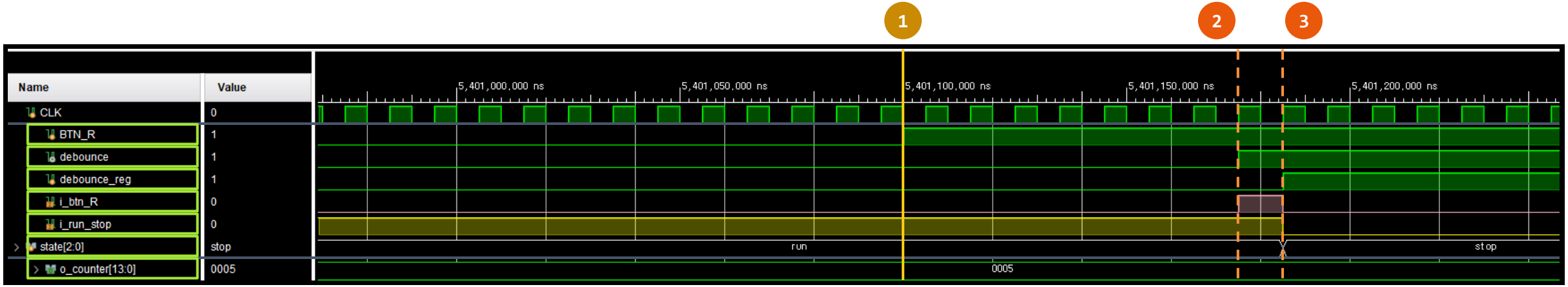


State 흐름
RUN : UP

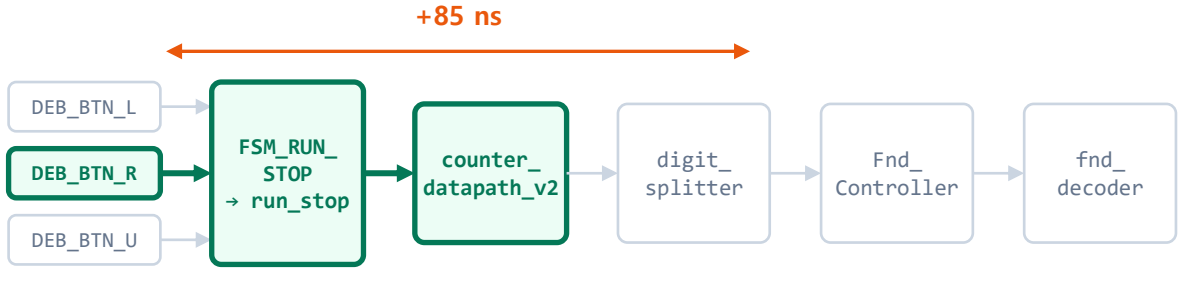
100 MHz debouncer

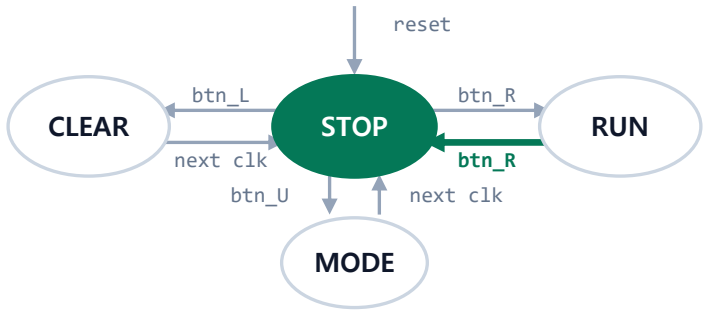
02

Second BTN_R Returns to STOP, Count Holds at 0005



1. **5,401,100 ns** 두 번째 BTN_R 입력
2. **5,401,175 ns** 8 sample 연속 1 → debounce · i_btn_R tick
3. **5,401,185 ns** RUN → STOP, i_run_stop = 0
4. **o_counter** 0005 유지 (STOP = tick 정지)



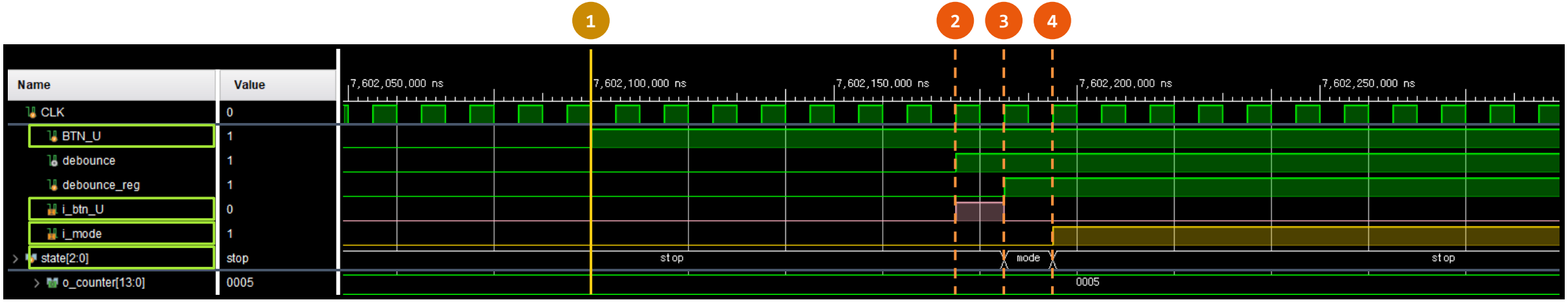


State 흐름
RUN → STOP

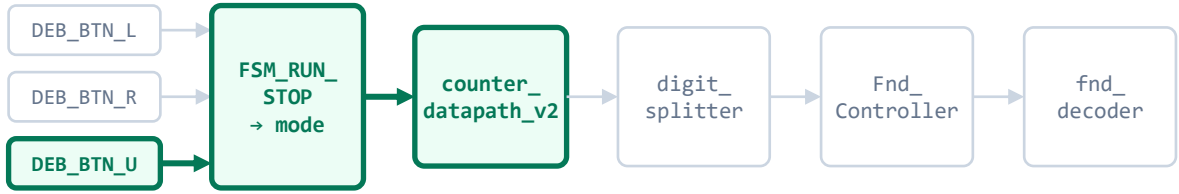
100 MHz debouncer

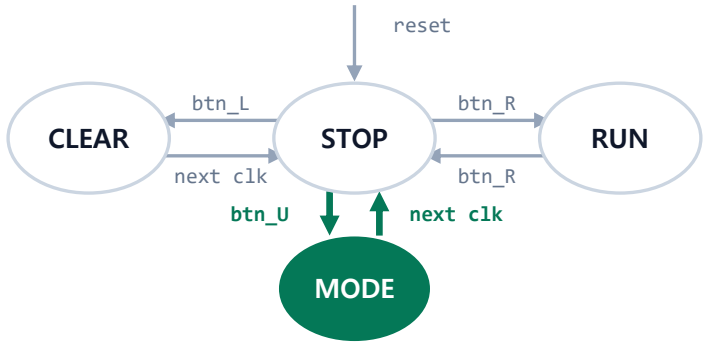
02

BTN_U Passes Through MODE, i_mode Flips on Return to STOP



1. **7,602,100 ns** BTN_U 입력
2. **+75 ns** i_btn_U tick
3. **+85 ns** STOP → MODE, mode_next = ~mode_reg
4. **+95 ns** MODE → STOP, mode_reg 갱신 → i_mode 1



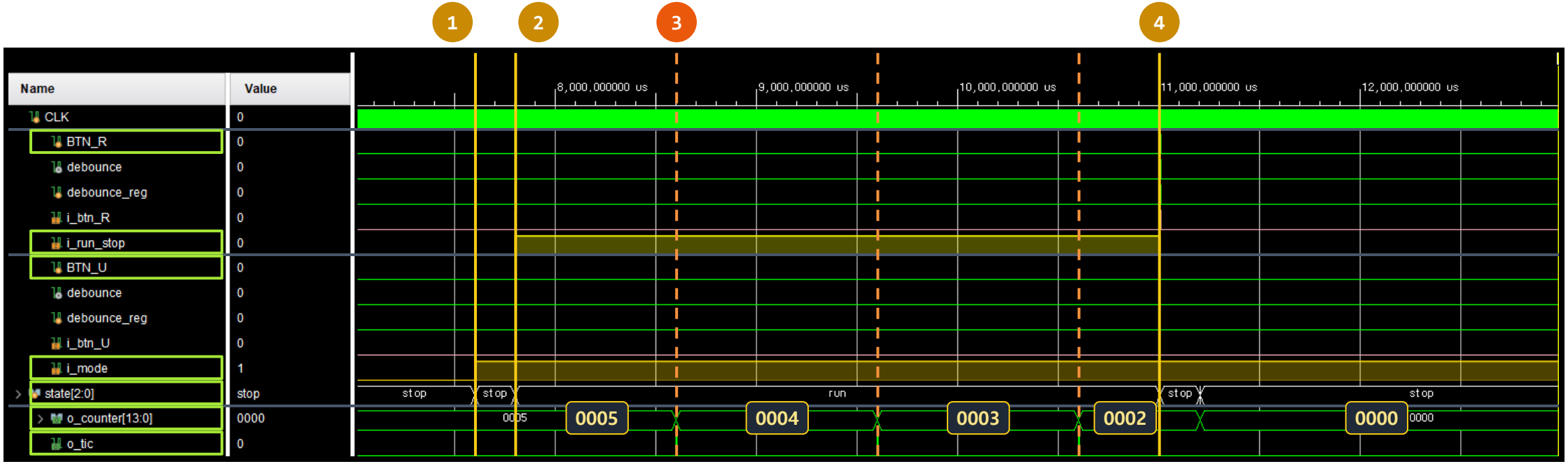


State 흐름
STOP → MODE → STOP

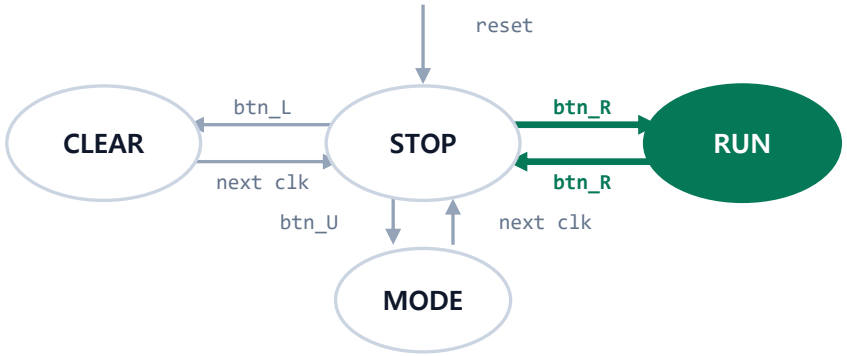
100 MHz debouncer

02

DOWN Mode Counts 0005 → 0002, Then BTN_R Stops



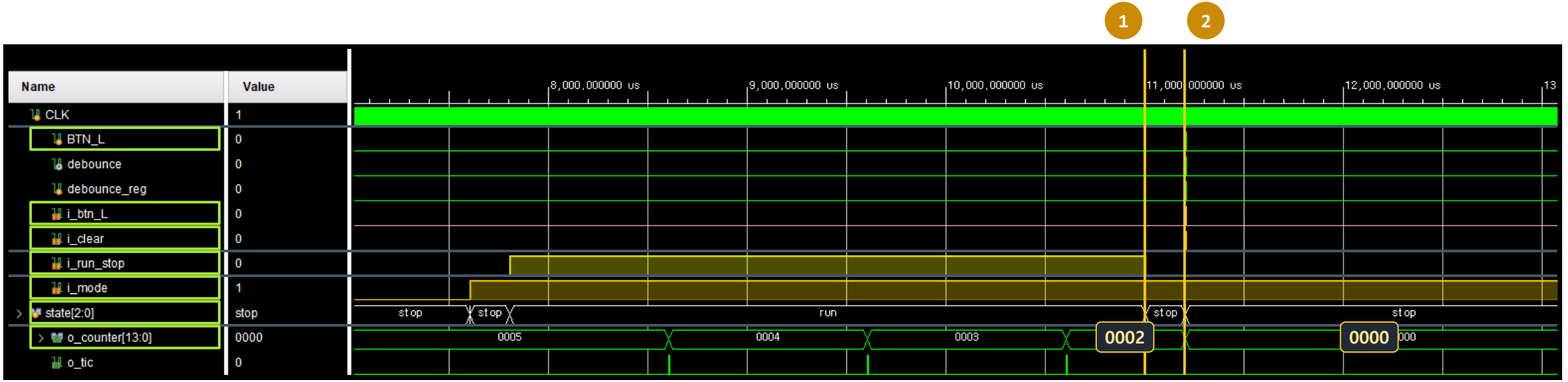
1. **7,602 µs** BTN_U → i_mode = 1 (DOWN)
2. **7,803 µs** BTN_R → RUN
3. **8.6 · 9.6 · 10.6 ms** tick마다 −1 : 0005 → 0004 → 0003 → 0002
4. **11,004 µs** BTN_R → STOP, 0002 유지



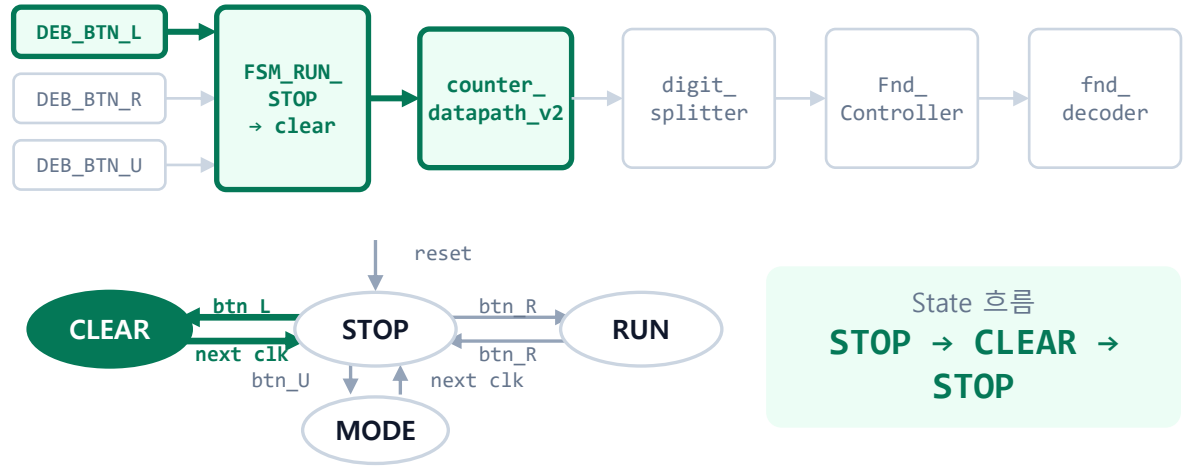
100 MHz debouncer

02

BTN_L in STOP Clears 0002 to 0000, Mode Stays DOWN



1. **11,004 µs** BTN_R → STOP (0002)
2. **11,205 µs** BTN_L → i_btn_L tick → i_clear pulse
3. **o_counter** 0002 → 0000, STOP 유지
4. **i_mode** 1 유지 (CLEAR 대상 = count)

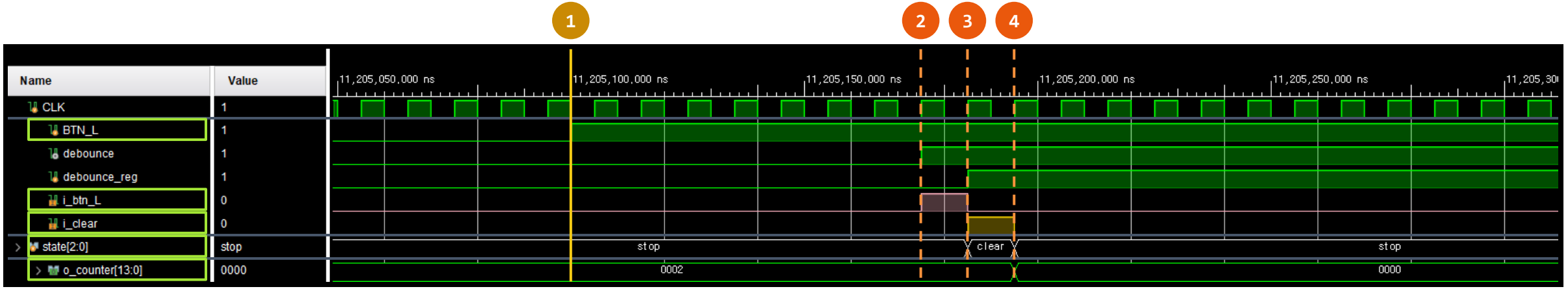


State 흐름
STOP → CLEAR → STOP

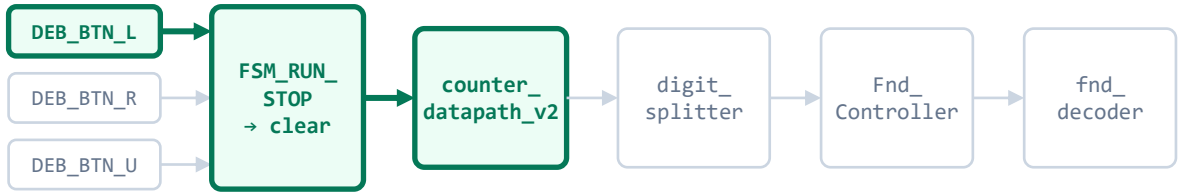
100 MHz debouncer

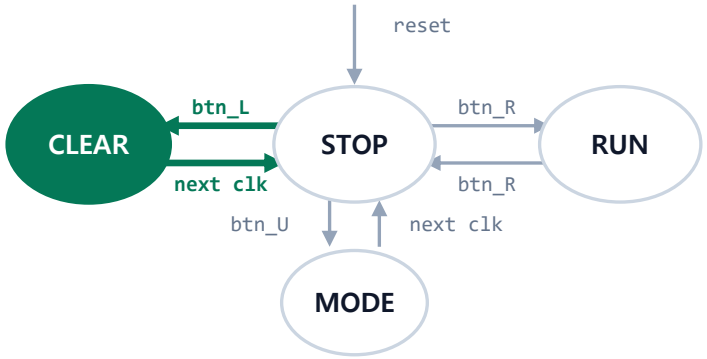
02

CLEAR Lasts One Clock and Resets o_counter at Its End



1. **11,205,100 ns** BTN_L 입력
2. **+75 ns** i_btn_L tick
3. **+85 ns** STOP → CLEAR, i_clear = 1 (1 clock)
4. **+95 ns** CLEAR → STOP, o_counter 0002 → 0000





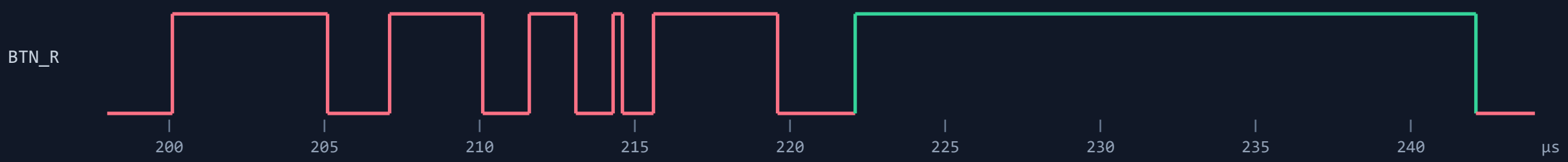
State 흐름
STOP → CLEAR → STOP

03

Scenario 2 · 입력 노이즈

Scenario 2: Noise Between 80 ns and 8 µs

driver(2) · 고정 noise table · 두 debouncer 동일 입력



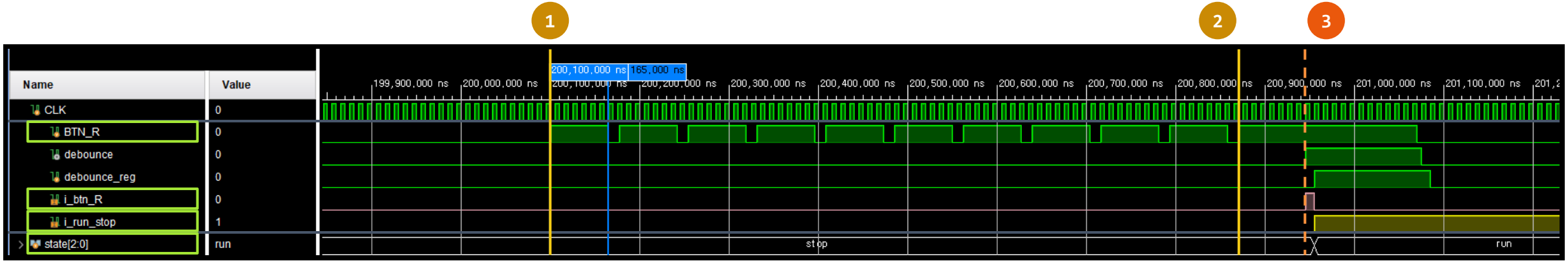
high 폭	5.0 µs	3.0 µs	1.5 µs	0.3 µs	4.0 µs	20 µs (press)
100 MHz (≥ 80 ns)	인정	인정	인정	인정	인정	인정
1 MHz (≥ 8 µs)	제거	제거	제거	제거	제거	인정

low 구간 1.0 ~ 2.5 µs (≥ 1 µs → 1 MHz sample 최소 1개 = 0)

100 MHz debouncer

03

Chatter Shorter Than 80 ns Never Fills q_reg



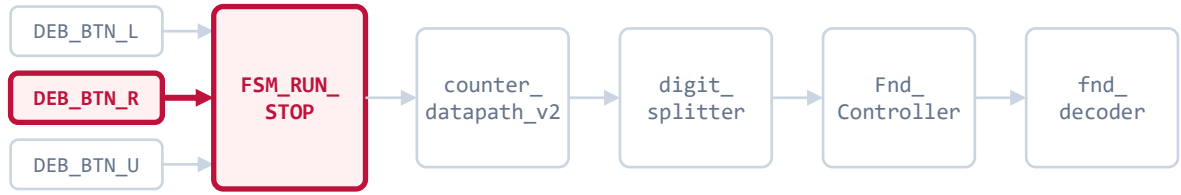
65 ns high / 12 ns low × 10

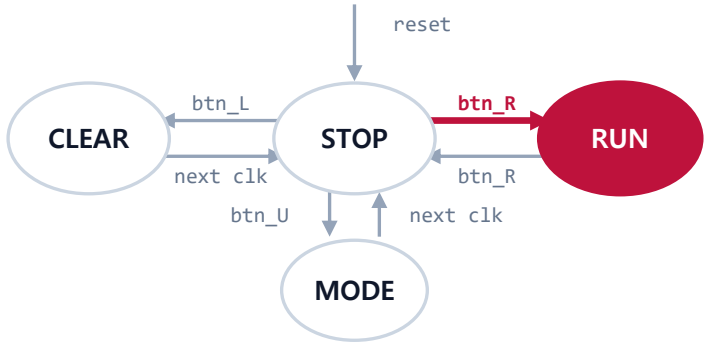
1 **200,100 ns** 65 ns high · 12 ns low 반복 시작

2 **high 65 ns** sample 최대 7개, low마다 q_reg에 0 유입

3 **200,871 ns** 안정 high 시작

4 **+75 ns** i_btn_R 1회 → STOP → RUN



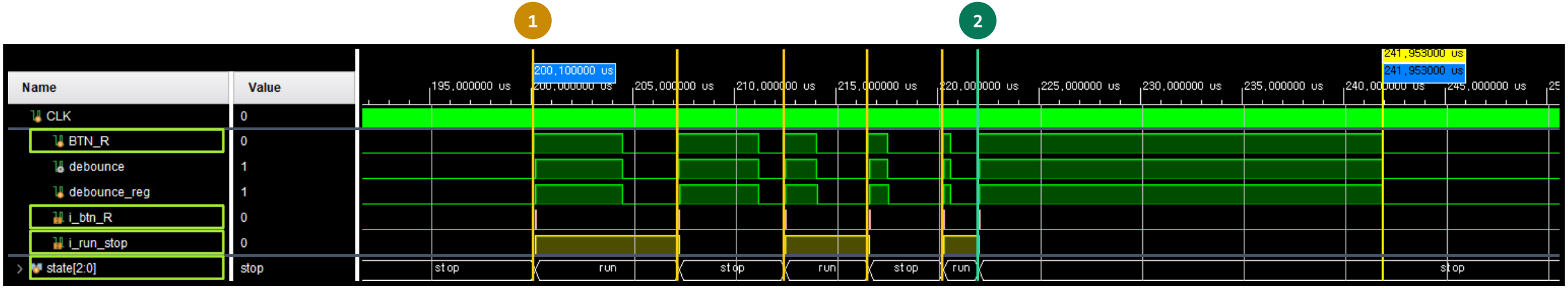


State 흐름
STOP → RUN ×1

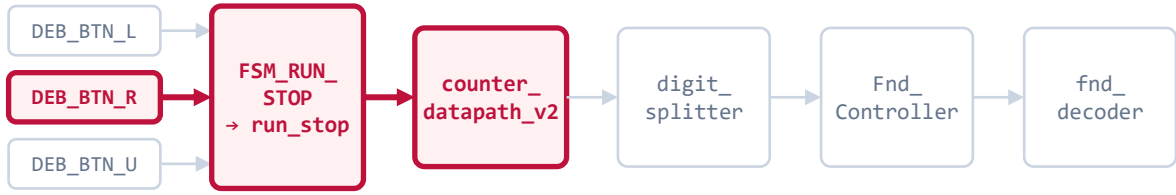
100 MHz debouncer

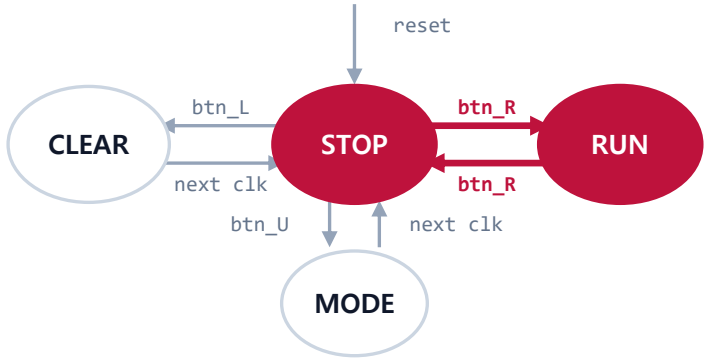
03

100 MHz Sampling Accepts Every Noise Burst as a Press



1. **200.1 ~ 219.6 µs** noise 5회 (0.45 ~ 4.4 µs) → 매번 tick
2. **222.1 µs** 실제 press → 6번째 tick
3. **state** RUN · STOP 6회 반전 → 최종 STOP
4. **결과** 의도 RUN, 실제 STOP (오동작)



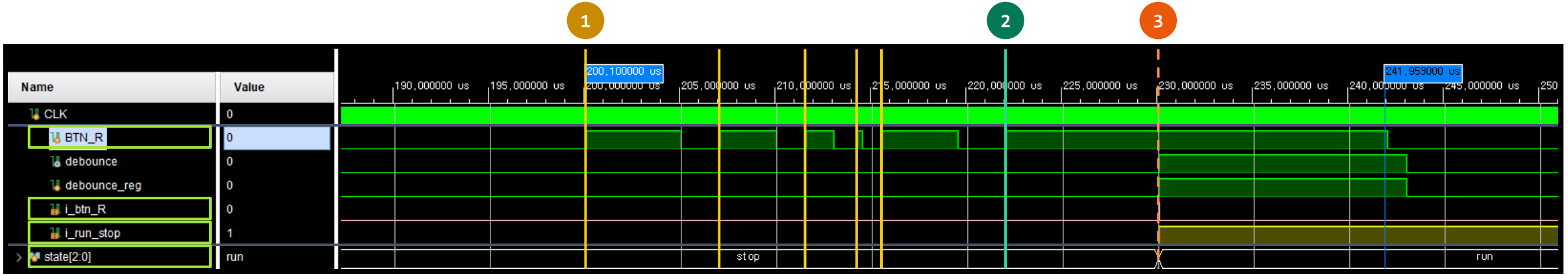


State 흐름
STOP ⇄ RUN ×6

1 MHz debouncer

03

1 MHz Sampling Rejects All Five Bursts and Starts RUN Once

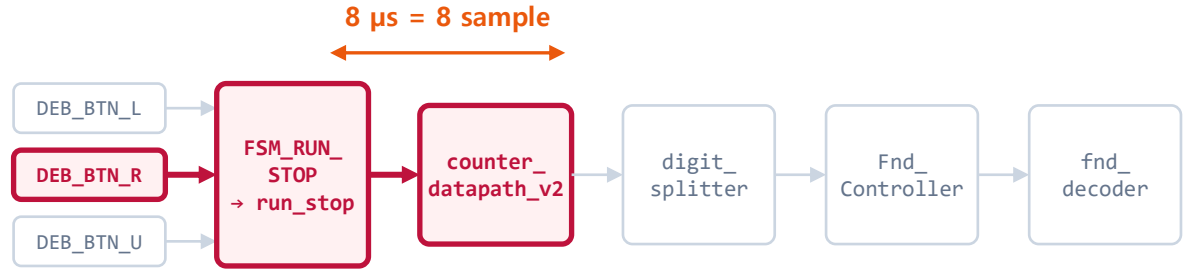


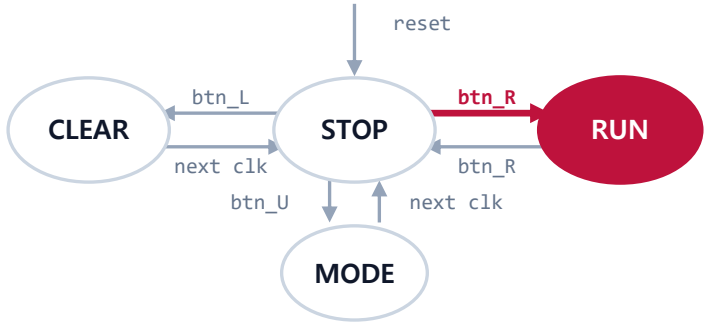
1 **200.1 ~ 219.6 µs** noise 5회 (0.3 ~ 5 µs) → tick 없음

2 **222.1 µs** 실제 press 시작

3 **230.1 µs** 8 sample 연속 1 → i_btn_R tick → RUN

4 **결과** RUN 1회, 의도대로 동작





State 흐름

STOP → RUN ×1

Same Noise: Six Toggles at 100 MHz, One at 1 MHz

03

입력 (BTN_R)	high 폭	button_debouncer (100 MHz)	button_debouncer_1MHz
noise burst ①	5.0 µs	tick → RUN	—
noise burst ②	3.0 µs	tick → STOP	—
noise burst ③	1.5 µs	tick → RUN	—
noise burst ④	0.3 µs	tick → STOP	—
noise burst ⑤	4.0 µs	tick → RUN	—
실제 press	20 µs	tick (+75 ns) → STOP	tick (+7,995 ns) → RUN
release noise ×2	3.0 µs	tick 2회 추가	—

1 **100 MHz** noise 5 + press 1 = tick 6 → 최종 STOP (오동작)

2 **1 MHz** noise 0 + press 1 = tick 1 → 최종 RUN

3 **근거** tb_counter_scaled driver(2), 두 debouncer 동일 고정 table 입력 (sim 실측 tick 수 · 시각)

Sampling Copies the Input Spectrum Every fs

04

① sampling 모델

$x_s(t) = x(t) \cdot \Sigma\, \delta(t - nT), \quad T = 1 / fs$

② 주파수 영역

$X_s(f) = fs \cdot \Sigma\, X(f - k \cdot fs)$

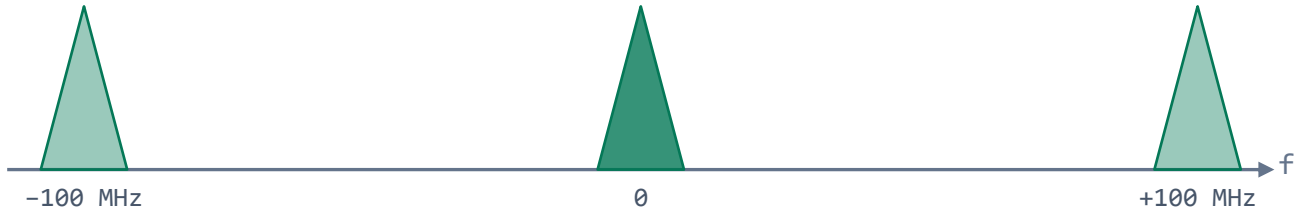
③ 의미

X(f) 복사본이 fs 간격으로 반복

④ Nyquist

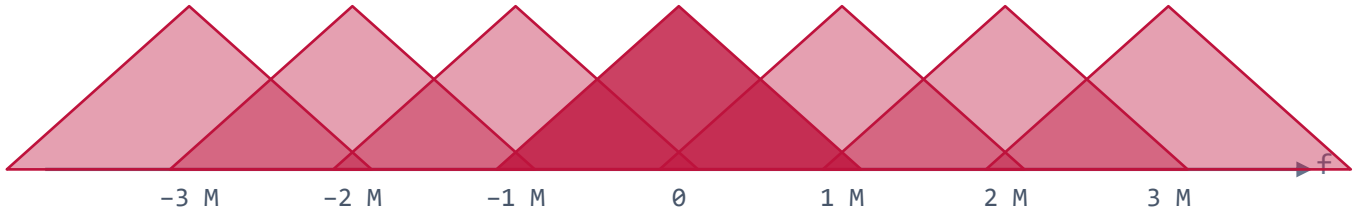
|f| < fs / 2 성분만 원형 보존
그 이상 = 0 ~ fs/2 로 folding (aliasing)

fs = 100 MHz



noise 대역 (≤ 3.3 MHz) ≪ fs/2 = 50 MHz → 복사본 간 겹침 없음, 원형 그대로 sampling

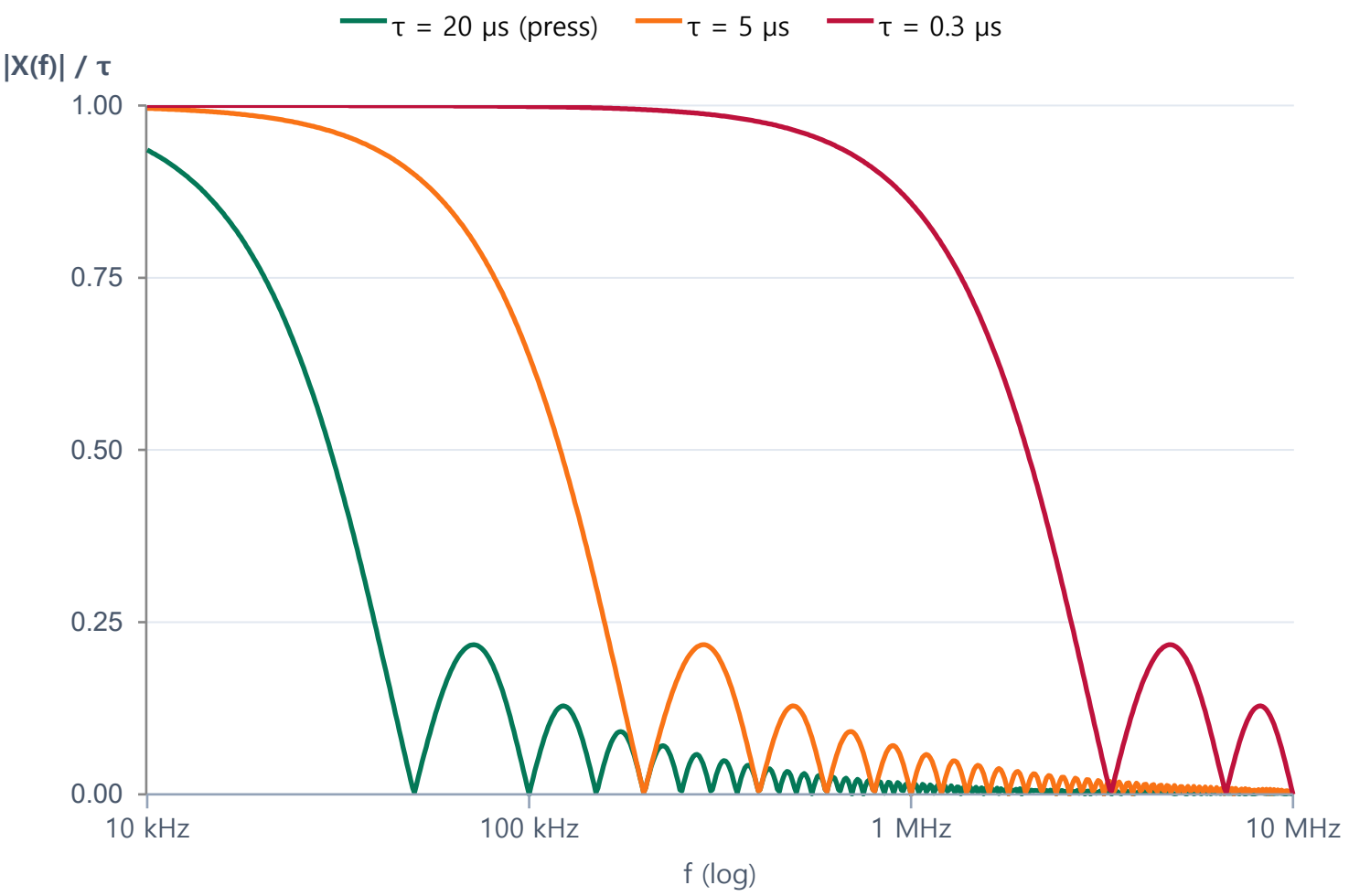
fs = 1 MHz



noise 대역 (0.2 ~ 3.3 MHz) > fs/2 = 500 kHz → 복사본 겹침, 0 ~ 500 kHz 로 folding

A Burst of Width τ Spreads Its Energy Up to 1/τ

04



$$x(t) = \mathrm{rect}(t / \tau)$$
$$|X(f)| = \tau \cdot |\sin(\pi f \tau) / (\pi f \tau)|$$

main lobe = 0 ~ 1/τ → burst 에너지 대부분

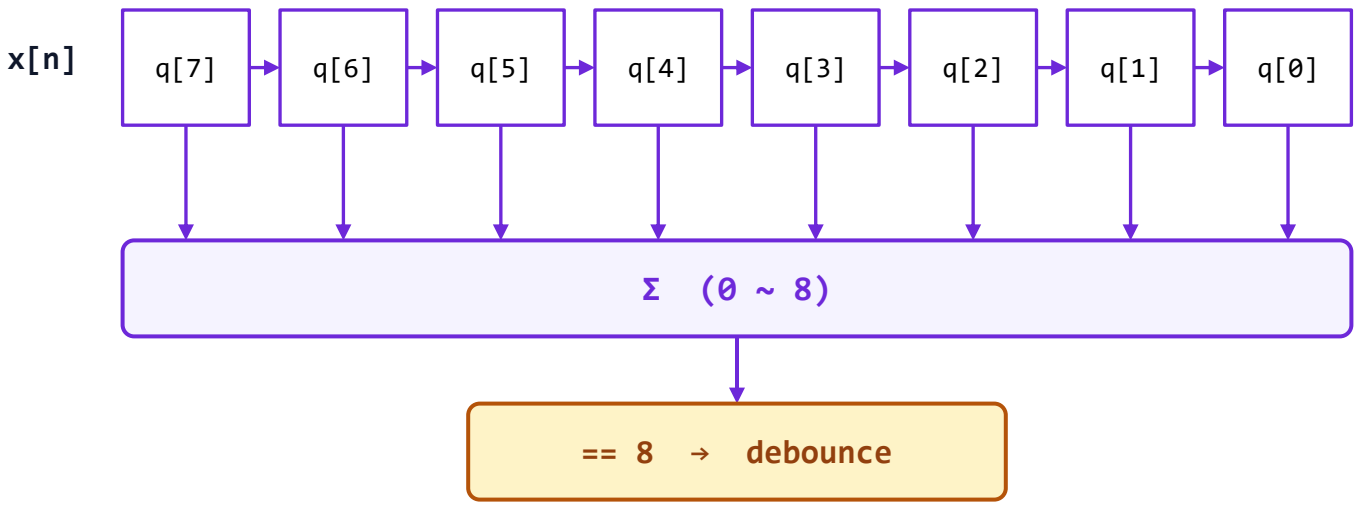
입력	τ	1/τ
noise ④	0.3 μs	3.33 MHz
noise ③	1.5 μs	667 kHz
noise ②	3.0 μs	333 kHz
noise ⑤	4.0 μs	250 kHz
noise ①	5.0 μs	200 kHz
실제 press	20 μs	50 kHz

1 **noise 대역** 0.2 ~ 3.3 MHz

2 **press 대역** ≤ 50 kHz

The 8-sample AND Is an 8-tap Moving Average with an 8/8 Threshold

04



8-input AND ≡ 8개 합 = 8

① debounce = 1 ⇔ Σ q[k] = 8 (k = 0 ~ 7)

② $s[n] = (1/8) \cdot \Sigma\, x[n-k]$ → 8-tap boxcar FIR

③ $H(f) = (1/8) \cdot \Sigma\, e^{-j2\pi fkT}$

④ $|H(f)| = \left| \sin(8\pi f/fs) / (8 \cdot \sin(\pi f/fs)) \right|$

⑤ |H(0)| = 1 · 첫 null fs/8 · −3 dB 0.0557·fs

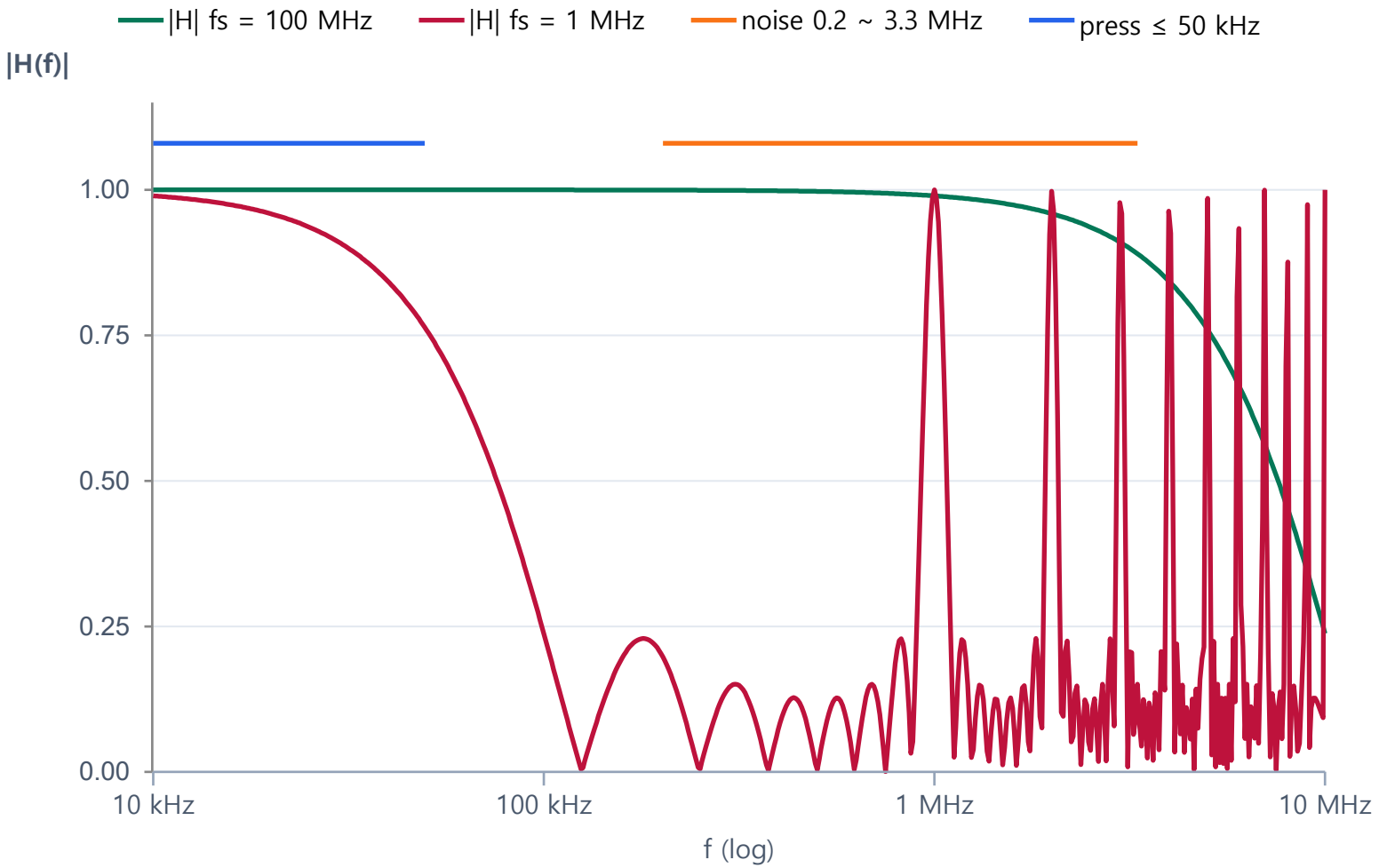
⑥ threshold 8/8 → s[n] = 1.0 (full scale)일 때만 tick

1 **선형부** moving average → low-pass, 통과 대역 ∝ fs/8

2 **비선형부** 문턱 8/8 → 통과 대역 밖 성분 = 부분 합 → tick 불가

Noise Sits in the 100 MHz Passband, Beyond the 1 MHz Cutoff

04



	100 MHz	1 MHz
−3 dB	5.57 MHz	55.7 kHz
첫 null	12.5 MHz	125 kHz
|H| 50 kHz	1.00	0.76
|H| 200 kHz	1.00	0.20
|H| 667 kHz	0.99	0.13
|H| 3.33 MHz	0.89	0.13

1 **100 MHz** noise 대역 |H| ≥ 0.89 → 통과

2 **1 MHz** noise 대역 |H| ≤ 0.20 → 문턱 미달

3 **1 · 2 · 3 MHz peak** |H| 주기 = fs → k·fs 성분 DC folding (p.27)

1/100 fs: 1/100 Passband, 100× Minimum Pulse (80 ns → 8 µs)

04

항목	fs = 100 MHz	fs = 1 MHz	관계
Sample 주기 T	10 ns	1 µs	T = 1 / fs
Window 8T	80 ns	8 µs	8 sample
−3 dB 대역	5.57 MHz	55.7 kHz	0.0557 · fs
첫 null	12.5 MHz	125 kHz	fs / 8
최소 인정 pulse	80 ns	8 µs	8 / fs
noise burst 5회 (0.3 ~ 5 µs)	5회 tick	0회	1/τ 대역 vs 통과 대역
실제 press 반응 시간	85 ns	8,005 ns	대가 = window 길이

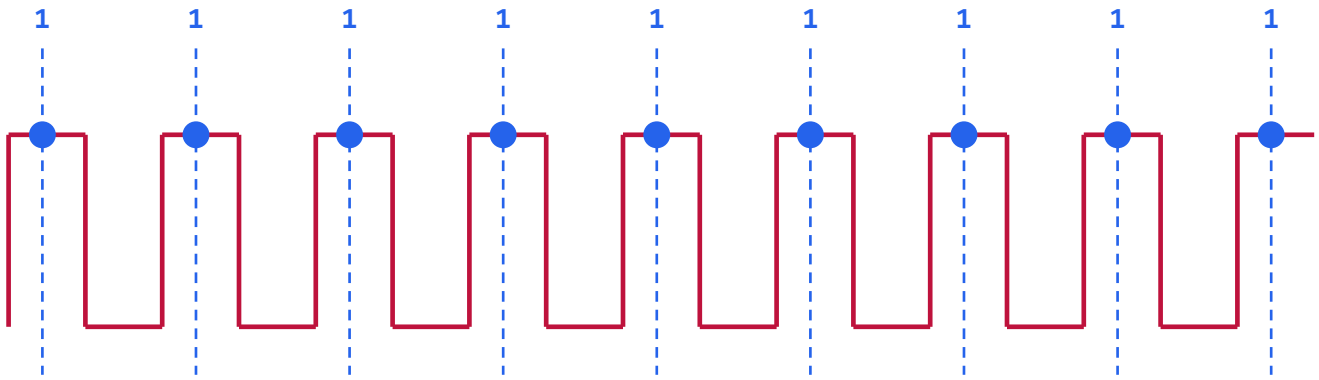
1 **시간 영역** pulse 폭 < 8T → 8 연속 1 불가 → 제거

2 **주파수 영역** 1/τ > 통과 대역 → |H| 감쇠 → 문턱 미달

3 **Trade-off** fs ↓ → noise 내성 ↑ · 반응 시간 ↑ (85 ns → 8,005 ns), 버튼 입력 = µs 지연 무관

Interference at k·fs Folds to DC and Passes Any Window

04



q_reg = 1111_1111 → debounce = 1 → 버튼 미입력 상태에서 tick

1. **주파수 해석** f = k·fs 성분 → sampling 후 f = 0 (DC)
2. **|H(k·fs)| = 1** window 길이 무관, 디지털 filter 단독 제거 불가

대응 (현업 구성)

1. **Anti-aliasing**
   보드 RC low-pass + Schmitt trigger → fs/2 이상 성분 sampling 전 제거
2. **Synchronizer**
   2-FF → 비동기 입력 metastability 차단
3. **Sample enable**
   i_clk 단일 clock + enable pulse → clock domain 1개 유지
4. **ms window**
   실제 bounce (ms 단위) 대응 → window = ms 단위

05

Scenario 3 · 버튼 우선순위

Scenario 3: Simultaneous Presses Follow R > L > U

driver(3) · press 20 µs · case a ~ f

Case	시작 State	입력	기대 결과
a	STOP	R + L + U 동시	R 우선 → RUN, L · U 무시
b	RUN	L + U 동시	RUN에서 무시 → RUN 유지
c	RUN	R + L 동시	R → STOP, L 무시 (count 유지)
d	STOP	L + U 동시	L 우선 → CLEAR, U 무시
e	STOP (count > 0)	L 먼저, R 30 ns 뒤	debouncer sampling 주기 의존
f	STOP (count > 0)	L 먼저, R 2 µs 뒤	순차 처리 → CLEAR 후 RUN

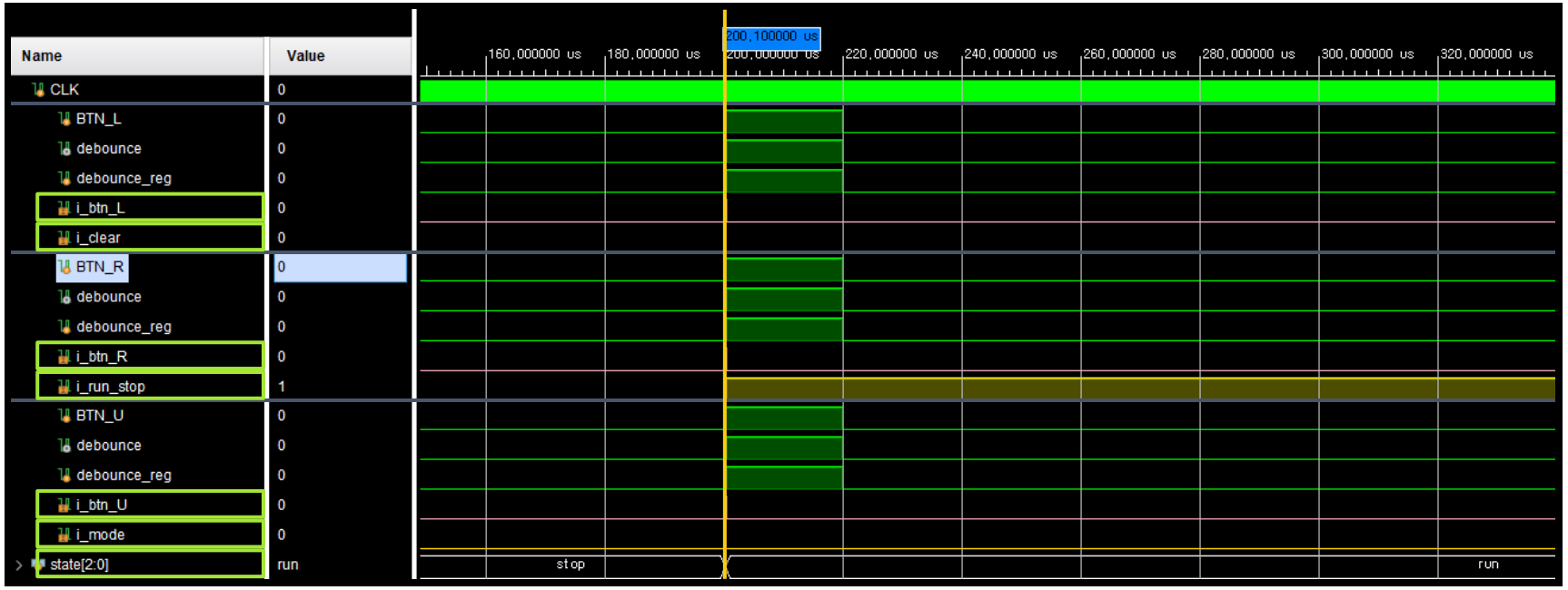
`STOP : if btn_R → else if btn_L → else if btn_U` `RUN : btn_R만 검사`

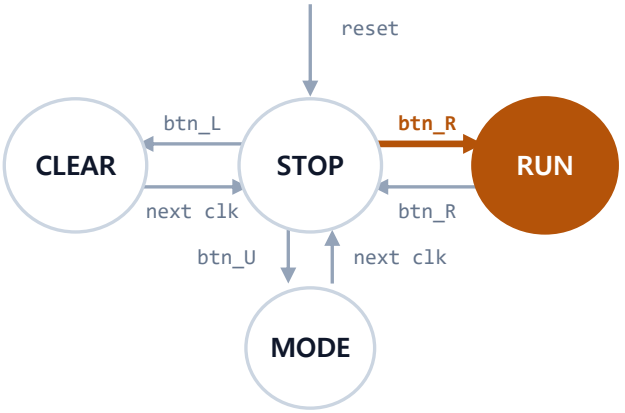
100 MHz debouncer

05

Case a: R + L + U in STOP, R Wins and Only RUN Is Entered

1





1 **200.1 µs** BTN_L · BTN_R · BTN_U 동시 20 µs

2 **tick** i_btn_L · i_btn_R · i_btn_U 같은 clock

3 **STOP** btn_R 우선 → RUN, i_clear · i_mode 변화 없음

tick R / L / U

1 / 1 / 1

결과 state

STOP → RUN

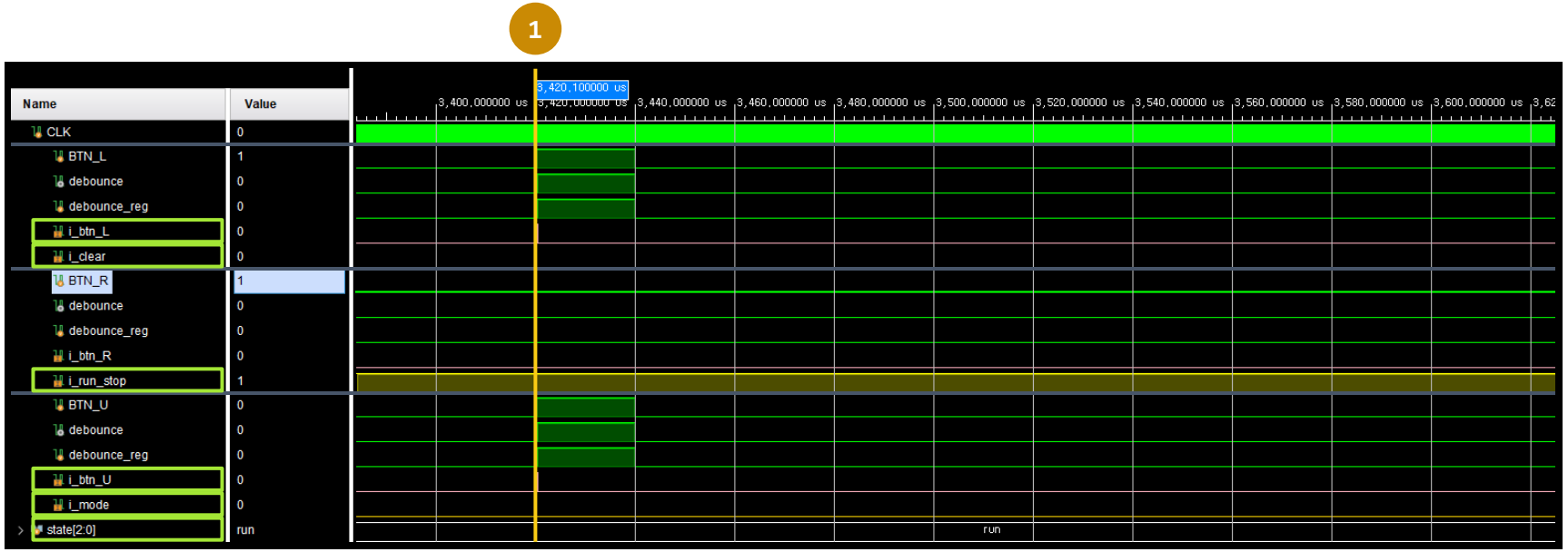
i_clear · i_mode

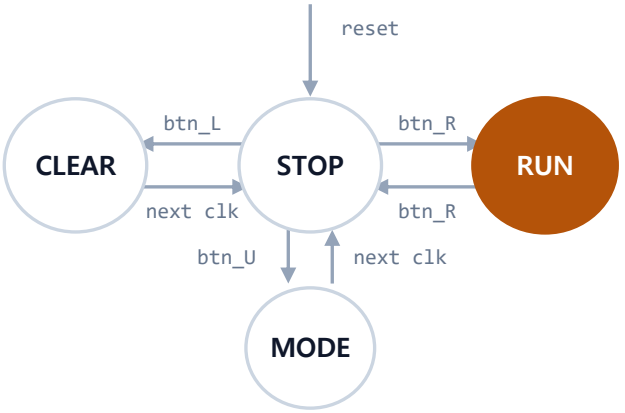
0 · 0 유지

100 MHz debouncer

05

Case b: L + U in RUN Are Ignored, Count Continues





1 **3,420.1 µs** RUN 중 BTN_L + BTN_U 동시

2 **tick** i_btn_L · i_btn_U 발생

3 **RUN** btn_R만 검사 → RUN 유지, i_clear · i_mode 0

tick R / L / U

0 / 1 / 1

결과 state

RUN 유지

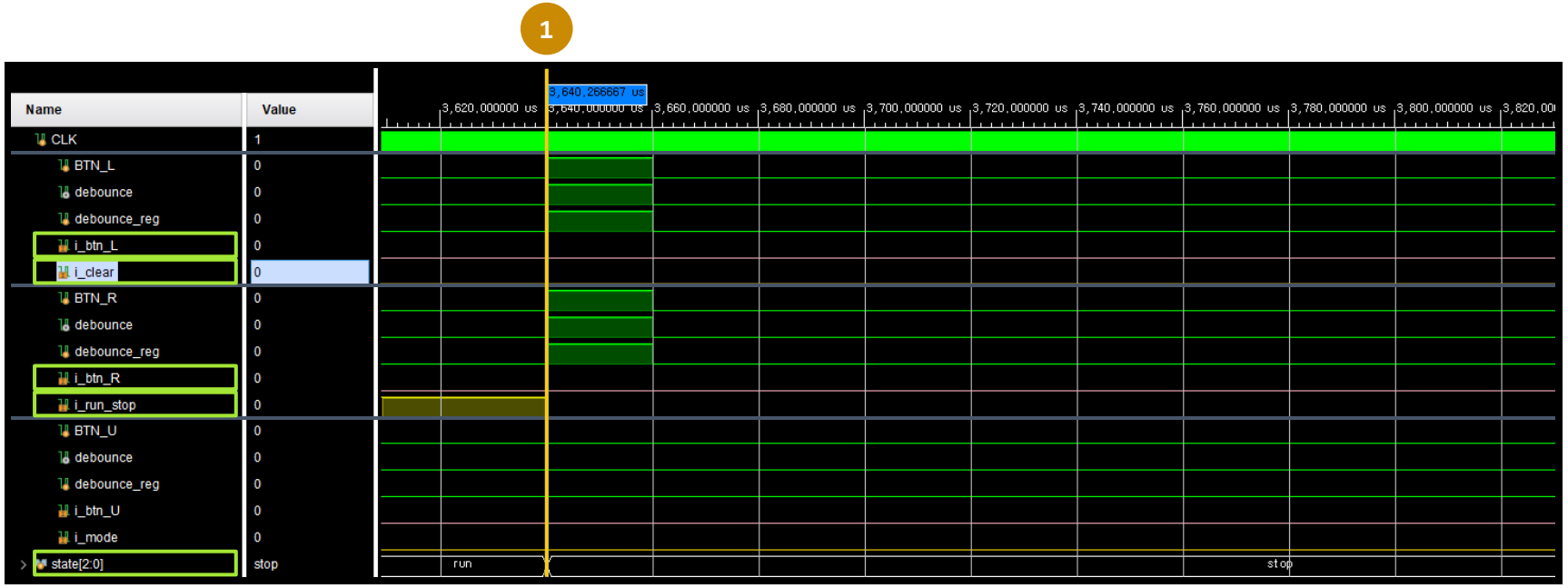
i_clear · i_mode

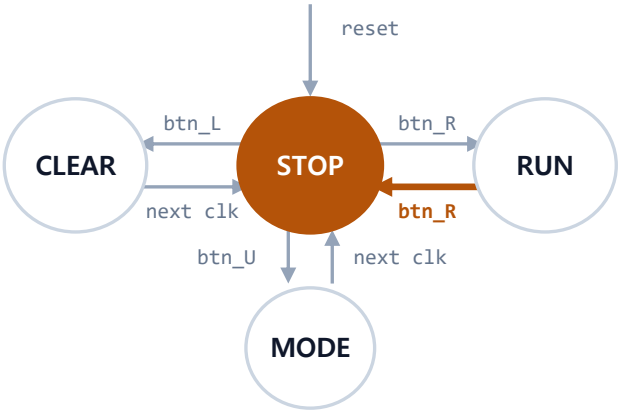
0 · 0 유지

100 MHz debouncer

05

Case c: R + L in RUN, STOP Wins and No Clear Occurs





1 **3,640.1 µs** RUN 중 BTN_L + BTN_R 동시

2 **RUN** btn_R → STOP

3 **btn_L** RUN에서 무시 → i_clear 0, count 유지

tick R / L / U

1 / 1 / 0

결과 state

RUN → STOP

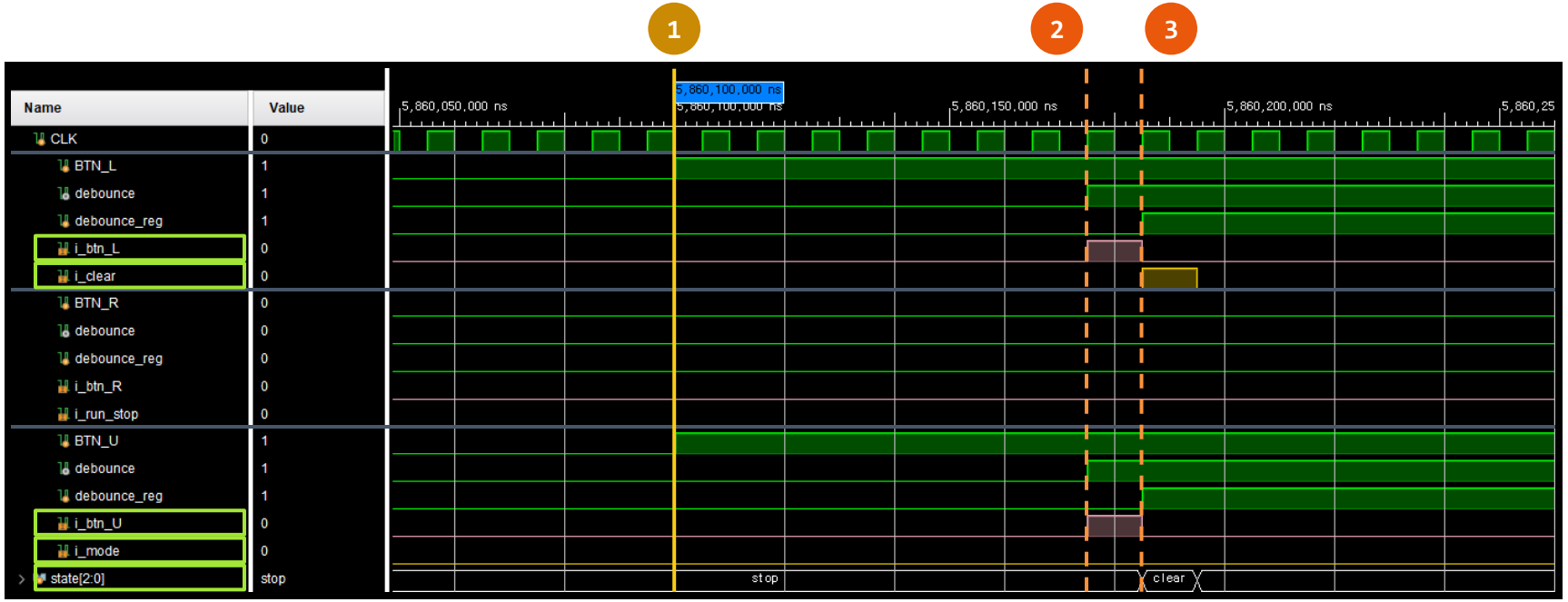
i_clear · count

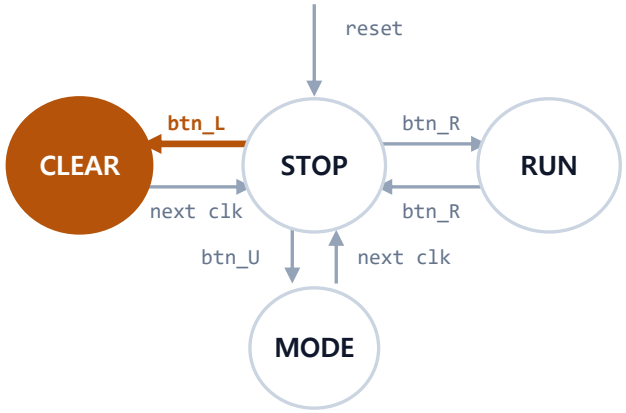
0 · 유지

100 MHz debouncer

05

Case d: L + U in STOP, CLEAR Wins and MODE Is Dropped





1 **5,860,100 ns** STOP 중 BTN_L + BTN_U 동시

2 **+75 ns** i_btn_L · i_btn_U 같은 clock tick

3 **+85 ns** btn_L 우선 → CLEAR, i_mode 0 유지

tick R / L / U

0 / 1 / 1

결과 state

STOP → CLEAR

i_mode

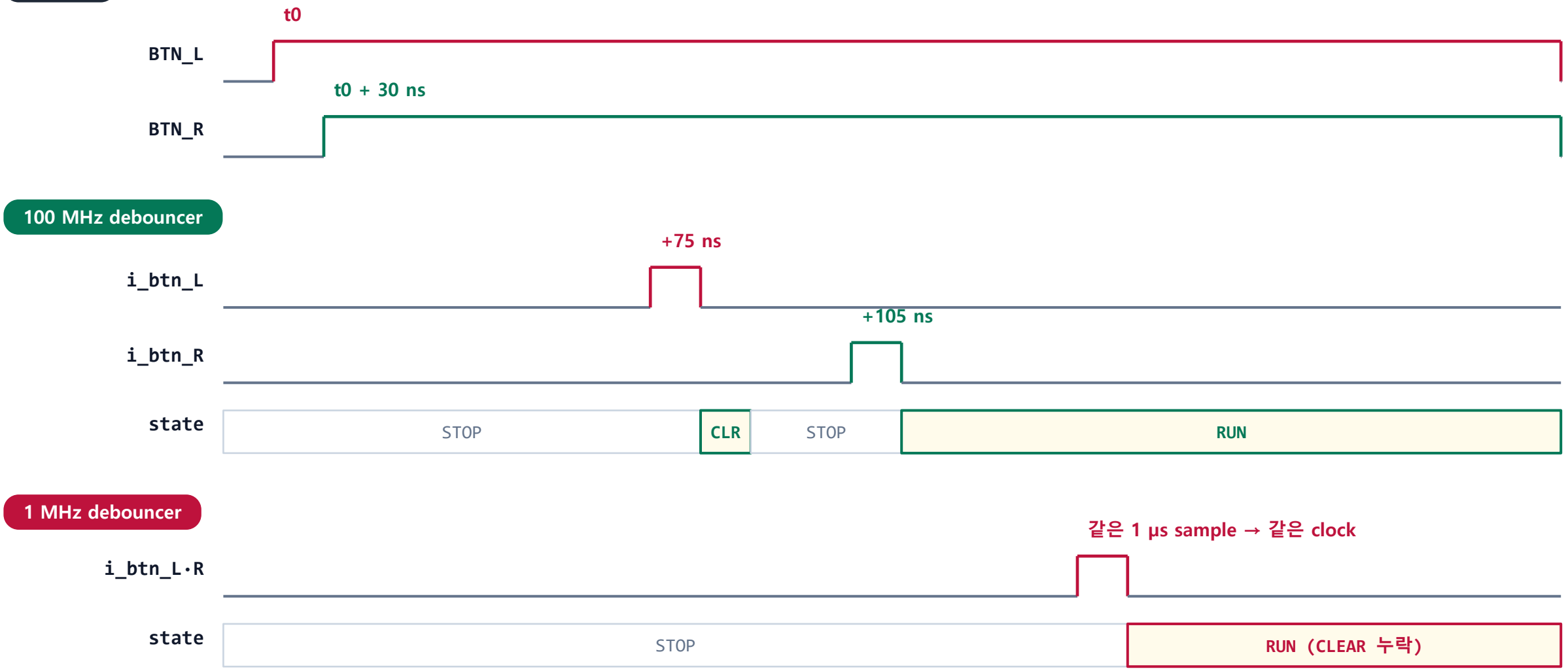
0 유지 (U 무시)

Case e: 30 ns Skew Makes Two Events at 100 MHz, One at 1 MHz

05

입력 (공통)

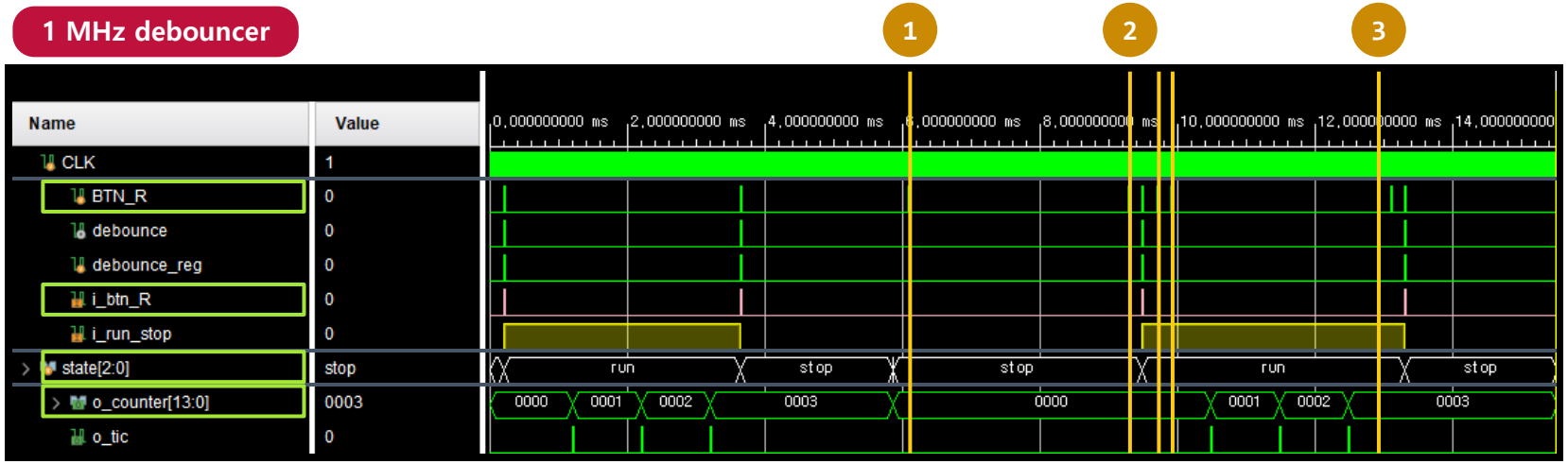
시간 축 개념도 (비례 아님)



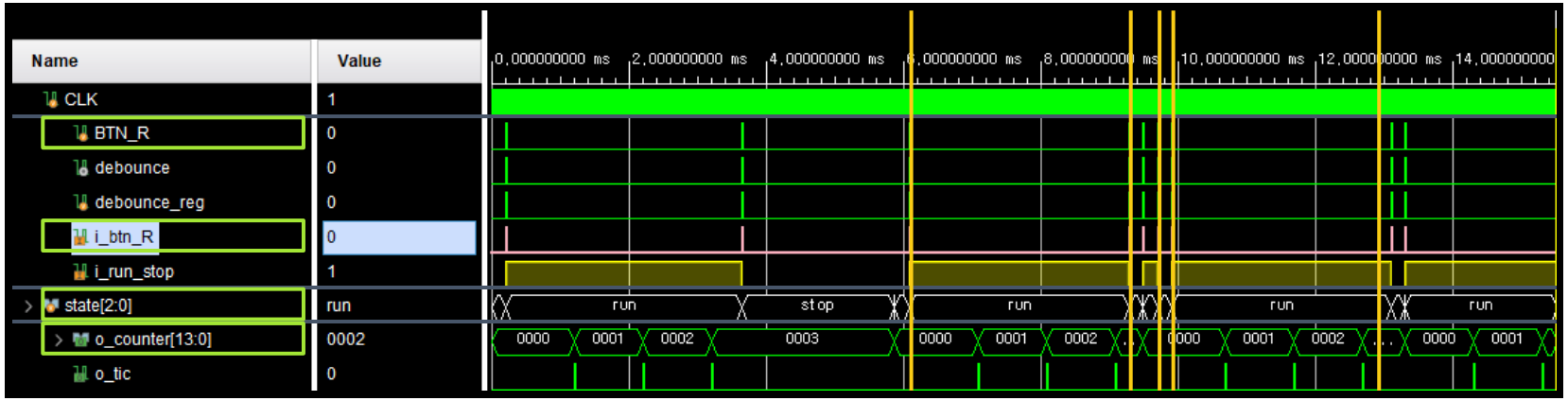
Same Stimulus Ends at 0003 with 1 MHz and 0002 with 100 MHz

05

1 MHz debouncer



100 MHz debouncer



1. **press() 폭 1 µs**
   capture 시점 TB 설정 (DEB_SCALE = 1)
   → 1 MHz window 8 µs 미달

2. **BTN_R 1 µs × 5**
   6.08 · 9.28 · 9.70 · 9.90 · 12.90 ms →
   100 MHz tick, 1 MHz 없음

3. **결과**
   1 MHz : 0003 · STOP
   100 MHz : 0002 · RUN

4. **원인**
   입력 data 동일, debouncer window 차이 → FSM 경로 분기

Both Debouncers Follow R > L > U, Except the 30 ns Skew Case

05

Case	입력	tick R / L / U	100 MHz 결과	1 MHz 결과	판정
a	STOP · R+L+U	1 / 1 / 1	RUN · clear 0 · mode 0	RUN · clear 0 · mode 0	R 우선
b	RUN · L+U	0 / 1 / 1	RUN 유지 · count 3	RUN 유지 · count 3	무시
c	RUN · R+L	1 / 1 / 0	STOP · clear 0 · count 3	STOP · clear 0 · count 3	R 우선
d	STOP · L+U	0 / 1 / 1	CLEAR 1회 · count 0	CLEAR 1회 · count 0	L 우선
e	STOP · L, R+30 ns	1 / 1 / 0	CLEAR 후 RUN · count 0	RUN · CLEAR 누락 · count 3	상이
f	STOP · L, R+2 µs	1 / 1 / 0	CLEAR 후 RUN · count 0	CLEAR 후 RUN · count 0	순차

1 **근거** tb_counter_scaled driver(3), press 20 µs, 100 MHz · 1 MHz 각각 실행 (e · f 전제 = STOP · count 3)

2 **Case e** 30 ns 차이 = 100 MHz에서 3 clock, 1 MHz에서 같은 sample → 동시 입력 → R 우선 → CLEAR 누락

Conclusion

06

구분	결과
설계	button_debouncer ×3 → FSM_RUN_STOP (STOP hub 4 state) → counter_datapath_v2 → FND scan
Scenario 1	UP 0000 → 0005, DOWN 0005 → 0002, CLEAR → 0000 · 버튼 → state 85 ns (100 MHz)
MODE	mode_reg = register → MODE 다음 clock에서 반전, i_mode 변경 = state 복귀 시점
Scenario 2	80 ns ~ 8 µs noise : 100 MHz 5회 오인식 → 최종 STOP, 1 MHz 0회 → RUN 1회
주파수 해석	8-sample AND = 8-tap moving average + 문턱 8/8 → 통과 대역 0.0557·fs
	100 MHz : noise 대역 |H| ≥ 0.89 통과 · 1 MHz : |H| ≤ 0.20 → tick 불가
Scenario 3	STOP R > L > U · RUN btn_R만 → a ~ d, f 두 debouncer 동일
	30 ns skew : 1 MHz 같은 sample 병합 → CLEAR 누락

Discussion and Testbench Feedback

06

설계 고찰

1. **Window 길이**
   1 MHz window 8 µs → ms 단위 실제 bounce 대비 짧음, 보드 적용 시 ms 단위 window 필요

2. **Sample enable**
   clk_1Mhz = logic 생성 clock → i_clk + enable 구조가 timing 분석 · skew 측면 유리

3. **Aliasing**
   k·fs 주기 간섭 = DC → 보드 RC + Schmitt trigger 병행 필요

4. **동시 입력 정책**
   sampling 주기에 따라 skew 입력 결과 상이 → 우선순위 · 입력 queue 정책 명시 필요

TB 피드백

1. **난수 noise**
   $urandom_range → 두 debouncer 입력 불일치 → 고정 table 교체로 비교 조건 동일화

2. **define 분리**
   `DEB_1MHZ가 TB · top 2곳 → 1 µs press + 1 MHz debouncer 조합 발생, 단일 정의 필요

3. **전제 조건**
   case e · f 시작 state (STOP · count > 0) 미검사 → 전제 붕괴 미검출

4. **자동 판정**
   $display 출력 → 기대값 비교 · PASS / FAIL · 오류 count 미구현